

Seat Control System (SCS) Edge Controller

Modelling, Simulation and Verification using MATLAB/Simulink

Project Report

Prepared by: **AMITH JOHN KOSHY**

Register No.: **25BEC0091**

Software: MATLAB / Simulink (R2025a model format)

September 2026

Contents

1. Introduction.....	3
1.1 Objective.....	3
1.2 Scope	3
2. System Description	3
2.1 Functional Requirements	3
2.2 Inputs and Outputs.....	3
3. Simulink Model.....	4
3.1 Model Overview	5
3.2 Block Description	6
3.3 Control Logic.....	6
3.4 Parameters and Simulation Settings	6
4. Work Carried Out.....	7
4.1 Model Construction.....	7
4.2 Issues Identified and Resolved	7
4.3 Automated Verification	7
4.4 Result Generation.....	7
4.5 Version Compatibility	7
5. Verification and Test Results	8
5.1 Test Cases	8
5.2 Test Script Output.....	8
6. Simulation Results	9
6.1 Scenario A - Nominal Operation.....	9
6.2 Scenario B - Fault Interlock.....	10
7. How to Run	10
7.1 Project Files	10
7.2 Steps	11
8. Conclusion	11
Appendix A: seat_control_scs_model.m.....	12
Appendix B: test_seat_control_scs.m	14
Appendix C: generate_figures.m.....	15

1. Introduction

1.1 Objective

The objective of this project is to model, simulate and verify the control logic of a Seat Control System (SCS) edge controller for a one-axis automotive power seat. The controller drives the seat forward or backward in response to driver switch inputs, stops the motion at the mechanical end positions using front and rear limit switches, disables the motor when a conflicting command is detected, and shapes the motor command with a comfort ramp so that the seat starts and stops smoothly.

1.2 Scope

- Development of a complete Simulink model of the controller and a simplified seat mechanism, built programmatically from a MATLAB script.
- Configuration of a fixed-step solver with a 10 ms step to represent the periodic scan cycle of an embedded controller.
- Automated verification of travel limits, fault detection and command slew rate using an assertion-based MATLAB test script.
- Simulation of a nominal operating sequence and a fault sequence, with results presented and analysed.
- Delivery of the model in MATLAB/Simulink R2025a file format.

2. System Description

The power seat moves along a single axis over a travel range of 0 mm (fully rearward) to 100 mm (fully forward). Two momentary switches, Forward and Backward, are operated by the driver. End-of-travel is detected by a front limit switch at 100 mm and a rear limit switch at 0 mm. The controller evaluates these inputs every scan cycle and produces a signed motor command.

2.1 Functional Requirements

ID	Requirement
R1	The seat shall move forward while the Forward switch is pressed and backward while the Backward switch is pressed.
R2	Seat travel shall be limited to 0 to 100 mm. Motion towards an end position that has already been reached shall be inhibited.
R3	Pressing both switches at the same time shall be treated as a fault, and the motor shall be disabled for as long as the fault is present.
R4	The motor command shall change at no more than 2 units/s (comfort ramp), so that start and stop are smooth.
R5	The controller shall execute on a fixed 10 ms scan cycle.

2.2 Inputs and Outputs

Signal	Type	Description
Fwd_Switch	Input (0/1)	Driver forward switch
Bwd_Switch	Input (0/1)	Driver backward switch
Front_Limit_Sensor	Internal (boolean)	True when seat position is at or above 100 mm

Signal	Type	Description
Rear_Limit_Sensor	Internal (boolean)	True when seat position is at or below 0 mm
Motor command	Output (-1 to +1)	Signed, ramp-limited motor command (+1 forward, -1 backward)
Motor enable	Output (0/1)	True when motion in either direction is permitted
Fault flag	Output (0/1)	True when both switches are pressed together
Seat position	Output (mm)	Simulated seat position

3. Simulink Model

The model, named SeatControlSCS, is created entirely by the script `seat_control_scs_model.m`. The script creates a new model, sets the solver configuration, adds and parameterises every block, connects all signal lines, saves the model and runs the simulation. Because the model is generated from source, it can be rebuilt identically at any time. The complete model is shown in Figure 1 on the following page.

3.1 Model Overview

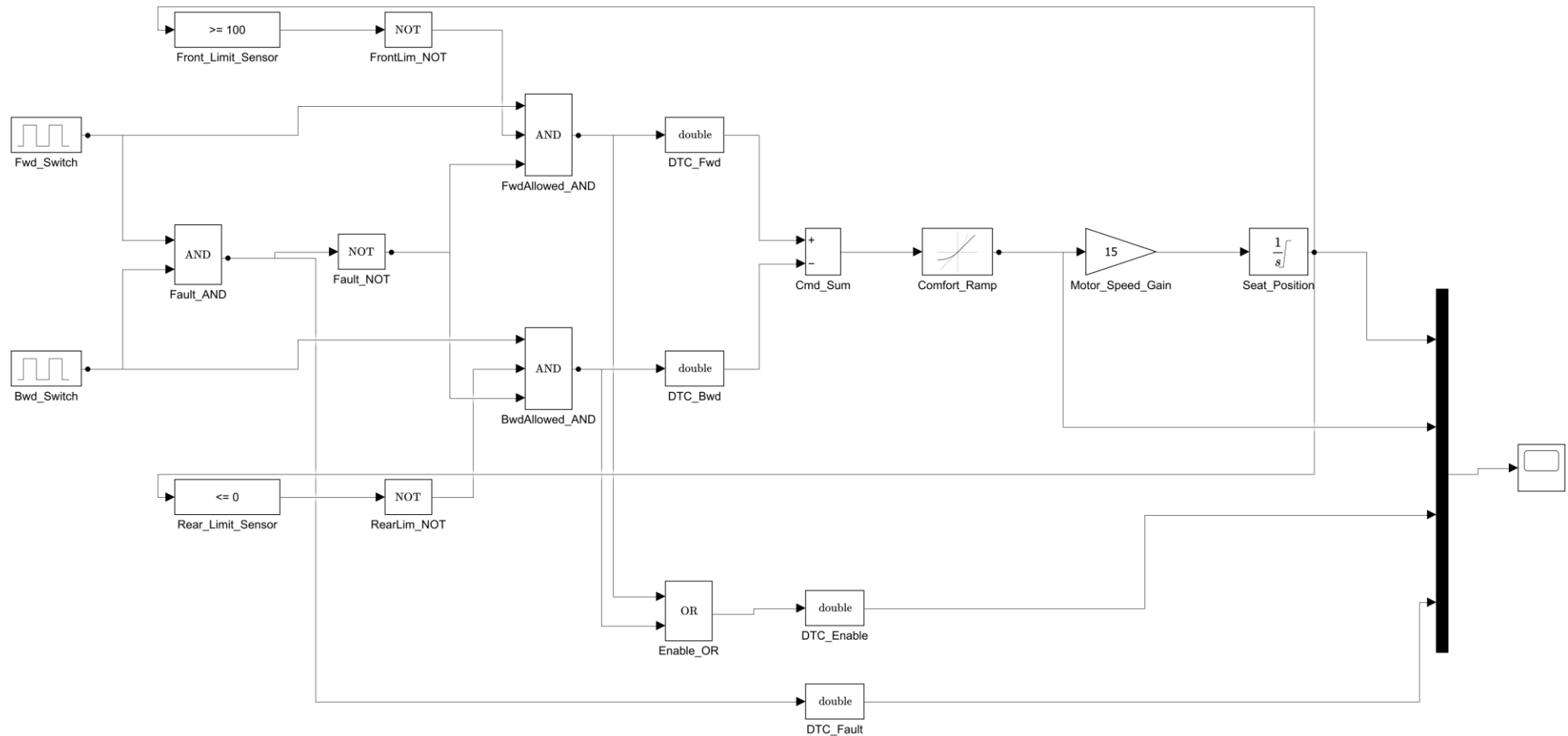


Figure 1: Simulink model *SeatControlSCS* (complete block diagram)

Signal flow runs from left to right: the driver switches and limit sensors feed the fault check and allowed-direction logic, which produce a signed command. The command passes through the comfort ramp and motor gain into the seat position integrator. The seat position is fed back to the limit sensors, closing the loop. Position, command, enable and fault are multiplexed to a Scope for observation.

3.2 Block Description

Block	Simulink library block	Function
Fwd_Switch, Bwd_Switch	Pulse Generator	Simulate driver switch presses (period 30 s; forward 30 % width from 0 s; backward 20 % width from 15 s)
Front_Limit_Sensor	Compare To Constant (≥ 100)	Front end-of-travel detection
Rear_Limit_Sensor	Compare To Constant (≤ 0)	Rear end-of-travel detection
FrontLim_NOT, RearLim_NOT	Logical Operator (NOT)	Convert limit state into a "not at limit" permission
Fault_AND	Logical Operator (AND)	Fault when both switches are pressed
Fault_NOT	Logical Operator (NOT)	"No fault" permission
FwdAllowed_AND, BwdAllowed_AND	Logical Operator (AND, 3 inputs)	Direction permitted = switch pressed AND not at limit AND no fault
Enable_OR	Logical Operator (OR)	Motor enable when either direction is permitted
DTC_Fwd, DTC_Bwd	Data Type Conversion	Boolean to double for the command sum
Cmd_Sum	Sum (+ -)	Signed command: forward minus backward
Comfort_Ramp	Rate Limiter	Limits command rate to ± 2 units/s
Motor_Speed_Gain	Gain (15)	Converts command to seat speed in mm/s
Seat_Position	Integrator (limited 0 to 100)	Seat position in mm, with mechanical end stops
DTC_Enable, DTC_Fault	Data Type Conversion	Boolean to double so all Mux inputs share one data type
Mux, Scope	Mux (4 inputs), Scope	Display of position, command, enable and fault

3.3 Control Logic

The logic evaluated every scan cycle is:

- $\text{Fault} = \text{Fwd AND Bwd}$
- $\text{FwdAllowed} = \text{Fwd AND (NOT FrontLimit) AND (NOT Fault)}$
- $\text{BwdAllowed} = \text{Bwd AND (NOT RearLimit) AND (NOT Fault)}$
- $\text{Enable} = \text{FwdAllowed OR BwdAllowed}$
- $\text{Command} = \text{RateLimit}(\text{FwdAllowed} - \text{BwdAllowed}, \pm 2 \text{ per s})$
- $\text{Position} = \text{Integral}(15 \times \text{Command}) \text{ dt, limited to } [0, 100] \text{ mm}$

Because the fault term is included in both permission equations, a fault inhibits motion in both directions. Because each limit term is included only in its own direction, the seat can always be driven away from an end position that has been reached.

3.4 Parameters and Simulation Settings

Parameter	Value	Location
Travel range	0 to 100 mm	Seat_Position saturation limits; limit sensor thresholds
Speed gain	15 mm/s at full command	Motor_Speed_Gain
Comfort ramp slew rate	$\pm 2 / -2$ units/s (0.5 s from 0 to full)	Comfort_Ramp

Parameter	Value	Location
Solver	ode4 (Runge-Kutta), fixed step	Model configuration
Fixed step size	0.01 s (10 ms scan cycle)	Model configuration
Stop time	30 s	Model configuration

4. Work Carried Out

4.1 Model Construction

The model is constructed programmatically using the Simulink API functions `new_system`, `add_block`, `set_param`, `add_line` and `save_system`. Block positions are set explicitly so that the generated diagram is readable, and all connections use automatic line routing.

4.2 Issues Identified and Resolved

The initial version of the build script did not simulate. The following issues were identified from the MATLAB error messages and corrected:

#	Observed problem	Cause	Resolution
1	Simulation stopped with: "The FixedStepDiscrete solver cannot be used to simulate block diagram SeatControlSCS because it contains continuous states".	The Integrator and Rate Limiter are continuous-time blocks. The discrete-only solver cannot integrate continuous states.	Solver changed to ode4 with the same fixed 10 ms step. Execution remains periodic, as in an embedded scan cycle, while the continuous seat dynamics are integrated correctly.
2	Data type mismatch at Mux input 2 (boolean expected, double supplied).	Mux combined double signals (position, command) with boolean signals (enable, fault). A Mux output must have a single data type.	Data Type Conversion blocks DTC_Enable and DTC_Fault added to convert enable and fault to double before the Mux.
3	Block labels overlapped in the block diagram.	Blocks were placed too close together.	Block positions re-spaced. No change to functionality.

4.3 Automated Verification

An assertion-based test script, `test_seat_control_scs.m`, was written. It builds the model, enables signal logging on the relevant block outputs at run time, runs two simulations and checks the logged data with assert statements. Any failed check stops the script with a descriptive error message. The test cases and their results are given in Section 5.

4.4 Result Generation

The script `generate_figures.m` rebuilds the model, exports the block diagram (Figure 1), runs the nominal and fault scenarios, and plots all key signals (Figures 2 and 3). It also prints the key numerical results quoted in Section 6.

4.5 Version Compatibility

The model file `SeatControlSCS.slx` is saved in MATLAB/Simulink R2025a format using `Simulink.exportToVersion`, so it opens directly in R2025a. The scripts use only standard functions available in earlier releases. The one theme-related call in `generate_figures.m` (theme) is executed only on releases that

support it, using `isMATLABReleaseOlderThan`. Running `seat_control_scs_model.m` in any supported release rebuilds the model natively in that release.

5. Verification and Test Results

5.1 Test Cases

Scenario A uses the default switch timing (the switches never overlap). Scenario B changes the phase delay of `Bwd_Switch` to 0 s so that both switches are pressed together during the first 6 s, which exercises the fault interlock.

ID	Scenario	Check	Pass criterion	Result
T1	A - Nominal	Travel range	$0 \leq \text{position} \leq 100 \text{ mm}$ at every step	PASS (min 0.00, max 100.00 mm)
T2	A - Nominal	Comfort ramp	$ \text{change in command} \leq 2 \times 0.01 = 0.02$ per step	PASS (max 0.0200)
T3	B - Fault	Fault detection	Fault flag = (Fwd AND Bwd) at every step, and the fault occurs at least once	PASS (fault active 6.01 s)
T4	B - Fault	Limits under fault	T1 and T2 criteria also hold during the fault scenario	PASS

5.2 Test Script Output

```
>> test_seat_control_scs
Model built and simulated: SeatControlSCS
PASS T1: position within [0, 100] mm (min 0.00, max 100.00)
PASS T2: max command change per step 0.0200 <= limit 0.0200
PASS T3: fault flag active exactly when both switches pressed (6.01 s)
PASS T4: range and slew limits hold during fault scenario
All SeatControlSCS tests passed.
```

All four test cases passed. The maximum command change of 0.0200 per step equals the configured limit exactly, which confirms that the comfort ramp is active and is the limiting factor during every start and stop.

6. Simulation Results

6.1 Scenario A - Nominal Operation

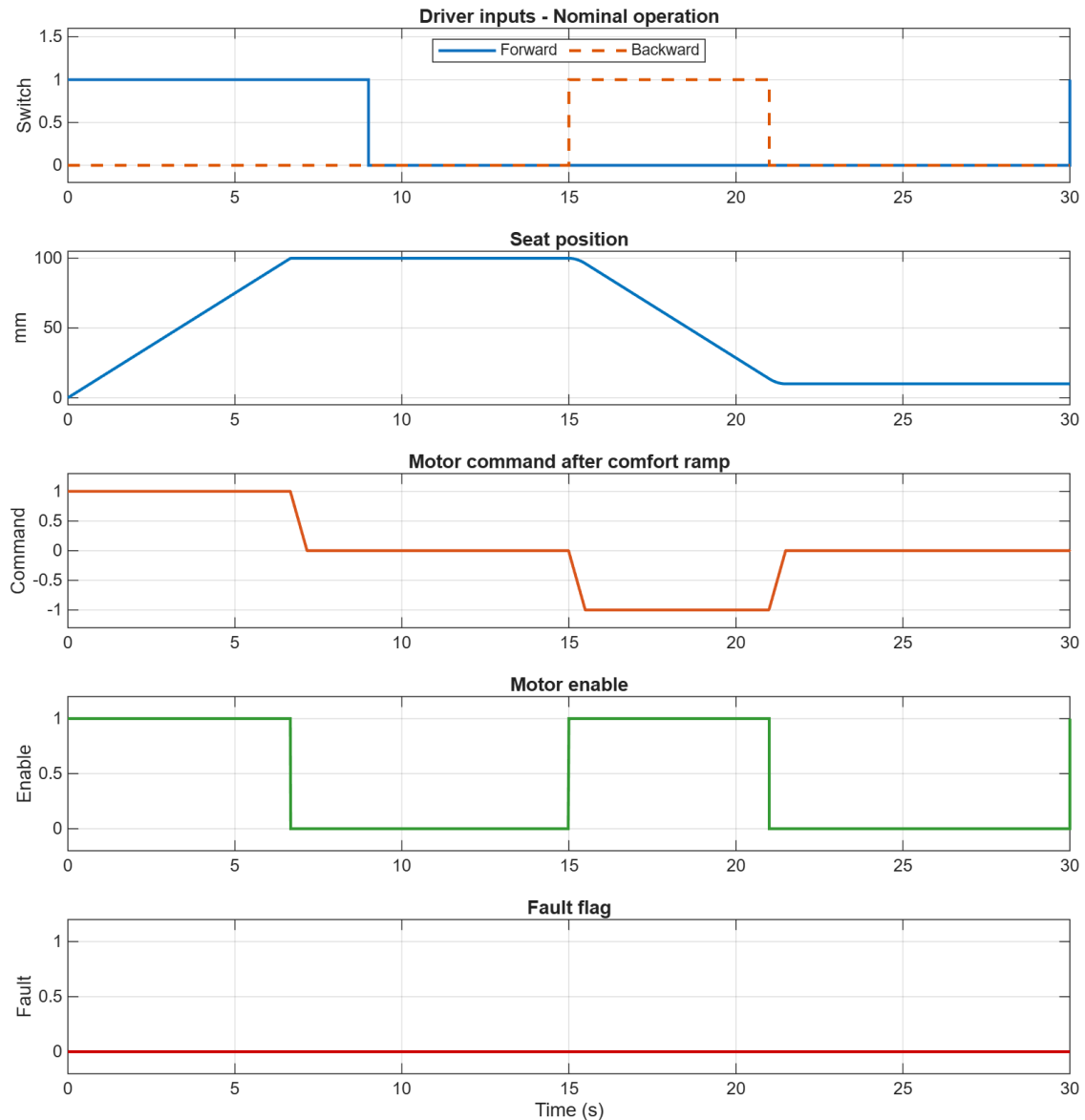


Figure 2: Nominal operation - switch inputs, seat position, motor command, enable and fault

- **0 to 9 s (Forward pressed):** the command ramps to +1 and the seat moves forward at 15 mm/s. It reaches the front limit of 100 mm at $t = 6.67$ s ($100 \text{ mm} / 15 \text{ mm/s}$). The front limit sensor removes forward permission, the enable drops to 0 and the command ramps down to 0 in 0.5 s. The position is held at 100 mm.
- **9 to 15 s (no switch pressed):** the seat remains stationary at 100 mm.
- **15 to 21 s (Backward pressed):** the command ramps to -1 in 0.5 s and the seat moves rearward at 15 mm/s. When the switch is released at 21 s the command ramps back to 0. The final position is 10.00 mm, a net travel of 90 mm ($6 \text{ s} \times 15 \text{ mm/s}$).
- **Fault flag:** remains 0 throughout, because the two switches are never pressed together.
- **$t = 30$ s:** the Forward pulse generator begins its next period, which appears as a rise in the switch and enable signals at the last sample.

6.2 Scenario B - Fault Interlock

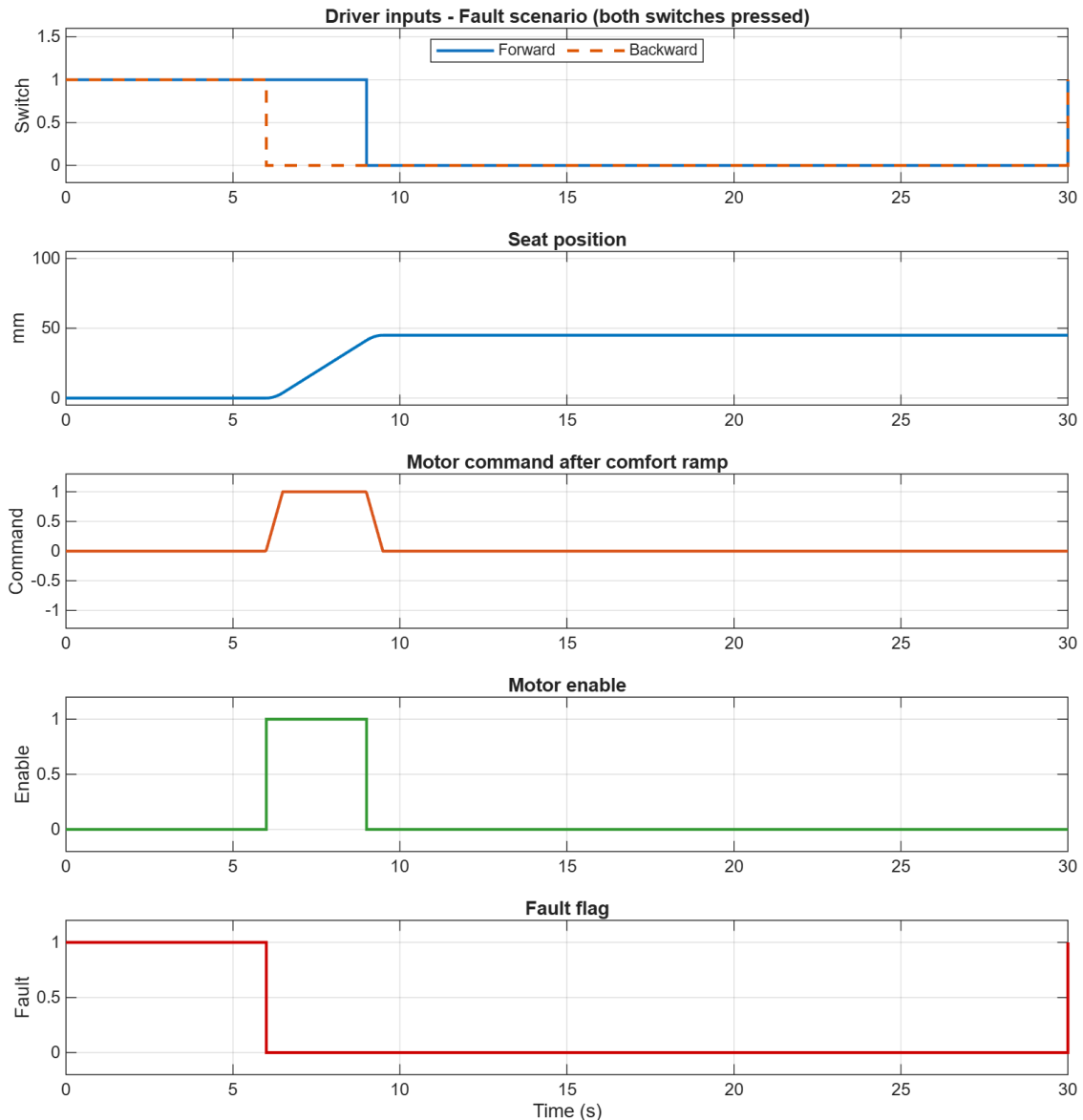


Figure 3: Fault scenario - both switches pressed together from 0 to 6 s

- **0 to 6 s (both switches pressed):** the fault flag is 1 for the full 6 s. Both direction permissions are removed, the enable stays at 0, the command stays at 0 and the seat remains at 0 mm.
- **6 to 9 s (Forward only):** the fault clears when Backward is released. Forward motion is permitted, the command ramps to +1 and the seat moves forward.
- **After 9 s:** Forward is released, the command ramps to 0 and the seat stops at 45.00 mm (3 s x 15 mm/s).

The result confirms requirement R3: a conflicting input disables the motor completely, and normal operation resumes as soon as the conflict is removed. Position and slew rate limits continue to hold during the fault.

7. How to Run

7.1 Project Files

File	Description
seat_control_scs_model.m	Builds, saves and simulates the SeatControlSCS model
test_seat_control_scs.m	Assertion-based verification (test cases T1 to T4)

File	Description
generate_figures.m	Exports the block diagram and the result plots to the figures folder
SeatControlSCS.slx	Simulink model, R2025a format
figures/	fig1_simulink_model.png, fig2_nominal_response.png, fig3_fault_response.png
SeatControlSCS_Report.docx / .pdf	This report

7.2 Steps

1. Open MATLAB and set the current folder to the project folder.
2. Build and simulate the model: `seat_control_scs_model`
3. Double-click the Scope block to view position, command, enable and fault.
4. Run the verification tests: `test_seat_control_scs`
5. Regenerate the figures: `generate_figures`

To observe the fault interlock interactively, open the `Bwd_Switch` block, set Phase delay to 0 and run the simulation again.

8. Conclusion

A Seat Control System edge controller for a one-axis power seat was modelled in Simulink and simulated on a 10 ms fixed-step scan cycle. Two build errors, a solver incompatible with continuous states and a data type mismatch at the Mux, were identified and corrected. An automated test script verifies that seat travel remains within 0 to 100 mm, that the fault flag is raised exactly when both switches are pressed and that the motor command never exceeds the comfort ramp slew rate. All tests passed. The simulation results show correct end-of-travel stopping at 6.67 s, smooth 0.5 s command ramps and a complete motor inhibit during the fault condition. The model is delivered in R2025a format together with the scripts needed to rebuild, test and plot it.

Appendix A: seat_control_scs_model.m

```

modelName = 'SeatControlSCS';

if bdIsLoaded(modelName)
    close_system(modelName, 0);
end
new_system(modelName);
open_system(modelName);

set_param(modelName, 'SolverType', 'Fixed-step');
set_param(modelName, 'Solver', 'ode4');
set_param(modelName, 'FixedStep', '0.01');
set_param(modelName, 'StopTime', '30');

add_block('simulink/Sources/Pulse Generator', [modelName '/Fwd_Switch'], ...
    'Position', [40 105 100 135], 'Amplitude', '1', 'Period', '30', ...
    'PulseWidth', '30', 'PhaseDelay', '0');

add_block('simulink/Sources/Pulse Generator', [modelName '/Bwd_Switch'], ...
    'Position', [40 305 100 335], 'Amplitude', '1', 'Period', '30', ...
    'PulseWidth', '20', 'PhaseDelay', '15');

add_block('simulink/Logic and Bit Operations/Compare To Constant', ...
    [modelName '/Front_Limit_Sensor'], 'Position', [180 15 270 45], ...
    'relop', '>=', 'const', '100');

add_block('simulink/Logic and Bit Operations/Compare To Constant', ...
    [modelName '/Rear_Limit_Sensor'], 'Position', [180 415 270 445], ...
    'relop', '<=', 'const', '0');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/Fault_AND'], 'Position', [180 195 220 245], ...
    'Operator', 'AND', 'Inputs', '2');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/Fault_NOT'], 'Position', [320 205 360 235], ...
    'Operator', 'NOT');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/FrontLim_NOT'], 'Position', [360 15 400 45], ...
    'Operator', 'NOT');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/RearLim_NOT'], 'Position', [360 415 400 445], ...
    'Operator', 'NOT');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/FwdAllowed_AND'], 'Position', [480 85 520 155], ...
    'Operator', 'AND', 'Inputs', '3');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/BwdAllowed_AND'], 'Position', [480 285 520 355], ...
    'Operator', 'AND', 'Inputs', '3');

add_block('simulink/Logic and Bit Operations/Logical Operator', ...
    [modelName '/Enable_OR'], 'Position', [600 500 640 550], ...
    'Operator', 'OR', 'Inputs', '2');

add_block('simulink/Signal Attributes/Data Type Conversion', ...
    [modelName '/DTC_Fwd'], 'Position', [600 105 650 135], ...
    'OutDataTypeStr', 'double');

add_block('simulink/Signal Attributes/Data Type Conversion', ...
    [modelName '/DTC_Bwd'], 'Position', [600 305 650 335], ...
    'OutDataTypeStr', 'double');

add_block('simulink/Math Operations/Sum', [modelName '/Cmd_Sum'], ...
    'Position', [720 200 750 240], 'Inputs', '+-', ...
    'IconShape', 'rectangular');

add_block('simulink/Discontinuities/Rate Limiter', [modelName '/Comfort_Ramp'], ...
    'Position', [820 200 880 240], 'RisingSlewLimit', '2', ...
    'FallingSlewLimit', '-2');

add_block('simulink/Math Operations/Gain', [modelName '/Motor_Speed_Gain'], ...

```

```

    'Position', [960 200 1020 240], 'Gain', '15');

add_block('simulink/Continuous/Integrator', [modelName '/Seat_Position'], ...
    'Position', [1100 200 1150 240], 'LimitOutput', 'on', ...
    'UpperSaturationLimit', '100', 'LowerSaturationLimit', '0');

add_block('simulink/Signal Attributes/Data Type Conversion', ...
    [modelName '/DTC_Enable'], 'Position', [720 510 770 540], ...
    'OutDataTypeStr', 'double');

add_block('simulink/Signal Attributes/Data Type Conversion', ...
    [modelName '/DTC_Fault'], 'Position', [720 590 770 620], ...
    'OutDataTypeStr', 'double');

add_block('simulink/Signal Routing/Mux', [modelName '/Mux'], ...
    'Position', [1260 250 1270 560], 'Inputs', '4');

add_block('simulink/Sinks/Scope', [modelName '/Scope'], ...
    'Position', [1330 385 1370 425]);
set_param([modelName '/Scope'], 'NumInputPorts', '1');
open_system([modelName '/Scope']);

add_line(modelName, 'Fwd_Switch/1', 'Fault_AND/1', 'autorouting', 'on');
add_line(modelName, 'Bwd_Switch/1', 'Fault_AND/2', 'autorouting', 'on');
add_line(modelName, 'Fault_AND/1', 'Fault_NOT/1', 'autorouting', 'on');

add_line(modelName, 'Front_Limit_Sensor/1', 'FrontLim_NOT/1', 'autorouting', 'on');
add_line(modelName, 'Rear_Limit_Sensor/1', 'RearLim_NOT/1', 'autorouting', 'on');

add_line(modelName, 'Fwd_Switch/1', 'FwdAllowed_AND/1', 'autorouting', 'on');
add_line(modelName, 'FrontLim_NOT/1', 'FwdAllowed_AND/2', 'autorouting', 'on');
add_line(modelName, 'Fault_NOT/1', 'FwdAllowed_AND/3', 'autorouting', 'on');

add_line(modelName, 'Bwd_Switch/1', 'BwdAllowed_AND/1', 'autorouting', 'on');
add_line(modelName, 'RearLim_NOT/1', 'BwdAllowed_AND/2', 'autorouting', 'on');
add_line(modelName, 'Fault_NOT/1', 'BwdAllowed_AND/3', 'autorouting', 'on');

add_line(modelName, 'FwdAllowed_AND/1', 'Enable_OR/1', 'autorouting', 'on');
add_line(modelName, 'BwdAllowed_AND/1', 'Enable_OR/2', 'autorouting', 'on');

add_line(modelName, 'FwdAllowed_AND/1', 'DTC_Fwd/1', 'autorouting', 'on');
add_line(modelName, 'BwdAllowed_AND/1', 'DTC_Bwd/1', 'autorouting', 'on');
add_line(modelName, 'DTC_Fwd/1', 'Cmd_Sum/1', 'autorouting', 'on');
add_line(modelName, 'DTC_Bwd/1', 'Cmd_Sum/2', 'autorouting', 'on');

add_line(modelName, 'Cmd_Sum/1', 'Comfort_Ramp/1', 'autorouting', 'on');
add_line(modelName, 'Comfort_Ramp/1', 'Motor_Speed_Gain/1', 'autorouting', 'on');
add_line(modelName, 'Motor_Speed_Gain/1', 'Seat_Position/1', 'autorouting', 'on');

add_line(modelName, 'Seat_Position/1', 'Front_Limit_Sensor/1', 'autorouting', 'on');
add_line(modelName, 'Seat_Position/1', 'Rear_Limit_Sensor/1', 'autorouting', 'on');

add_line(modelName, 'Seat_Position/1', 'Mux/1', 'autorouting', 'on');
add_line(modelName, 'Comfort_Ramp/1', 'Mux/2', 'autorouting', 'on');
add_line(modelName, 'Enable_OR/1', 'DTC_Enable/1', 'autorouting', 'on');
add_line(modelName, 'DTC_Enable/1', 'Mux/3', 'autorouting', 'on');
add_line(modelName, 'Fault_AND/1', 'DTC_Fault/1', 'autorouting', 'on');
add_line(modelName, 'DTC_Fault/1', 'Mux/4', 'autorouting', 'on');
add_line(modelName, 'Mux/1', 'Scope/1', 'autorouting', 'on');

save_system(modelName);
sim(modelName);

disp('Model built and simulated: SeatControlSCS');
disp('Open the Scope block to see seat position, motor command, enable, and fault signals.');
```

Appendix B: test_seat_control_scs.m

```

seat_control_scs_model;
mdl = 'SeatControlSCS';

sigs = { ...
    'Seat_Position', 'pos_log'; ...
    'Comfort_Ramp', 'cmd_log'; ...
    'Fault_AND', 'fault_log'; ...
    'Fwd_Switch', 'fwd_log'; ...
    'Bwd_Switch', 'bwd_log'};

for i = 1:size(sigs, 1)
    ph = get_param([mdl '/' sigs{i, 1}], 'PortHandles');
    set_param(ph.Outport(1), 'DataLogging', 'on', ...
        'DataLoggingNameMode', 'Custom', 'DataLoggingName', sigs{i, 2});
end
set_param(mdl, 'SignalLogging', 'on', 'SignalLoggingName', 'logout');

getData = @(logs, name) logs.getElement(name).Values.Data;

fixedStep = str2double(get_param(mdl, 'FixedStep'));
slewLimit = str2double(get_param([mdl '/Comfort_Ramp'], 'RisingSlewLimit'));
maxStepChange = slewLimit * fixedStep;

outA = sim(mdl);
logsA = outA.get('logout');
posA = getData(logsA, 'pos_log');
cmdA = getData(logsA, 'cmd_log');

assert(all(posA >= -1e-9) && all(posA <= 100 + 1e-9), ...
    'Seat position exceeded [0, 100] mm travel range. ');
fprintf('PASS T1: position within [0, 100] mm (min %.2f, max %.2f)\n', min(posA), max(posA));

assert(all(abs(diff(cmdA)) <= maxStepChange + 1e-9), ...
    'Motor command jumped by more than the configured slew rate in one step. ');
fprintf('PASS T2: max command change per step %.4f <= limit %.4f\n', max(abs(diff(cmdA))), maxStepChange);

set_param([mdl '/Bwd_Switch'], 'PhaseDelay', '0');
outB = sim(mdl);
logsB = outB.get('logout');
faultB = getData(logsB, 'fault_log');
fwdB = getData(logsB, 'fwd_log');
bwdB = getData(logsB, 'bwd_log');
posB = getData(logsB, 'pos_log');
cmdB = getData(logsB, 'cmd_log');

bothPressed = (fwdB > 0.5) & (bwdB > 0.5);
assert(any(bothPressed), ...
    'Overlap input failed to press both switches. ');
assert(isequal(logical(faultB), bothPressed), ...
    'Fault flag did not match "both switches pressed" condition. ');
fprintf('PASS T3: fault flag active exactly when both switches pressed (%.2f s)\n', sum(bothPressed) *
fixedStep);

assert(all(posB >= -1e-9) && all(posB <= 100 + 1e-9), ...
    'Seat position exceeded travel range during fault scenario. ');
assert(all(abs(diff(cmdB)) <= maxStepChange + 1e-9), ...
    'Motor command exceeded slew rate during fault scenario. ');
fprintf('PASS T4: range and slew limits hold during fault scenario\n');

disp('All SeatControlSCS tests passed.');
```

Appendix C: generate_figures.m

```

seat_control_scs_model;
mdl = 'SeatControlSCS';

figDir = fullfile(pwd, 'figures');
if ~exist(figDir, 'dir')
    mkdir(figDir);
end

print(['-s' mdl], '-dpng', '-r200', fullfile(figDir, 'fig1_simulink_model.png'));

sigs = { ...
    'Seat_Position', 'pos_log'; ...
    'Comfort_Ramp', 'cmd_log'; ...
    'Enable_OR', 'en_log'; ...
    'Fault_AND', 'fault_log'; ...
    'Fwd_Switch', 'fwd_log'; ...
    'Bwd_Switch', 'bwd_log'};

for i = 1:size(sigs, 1)
    ph = get_param([mdl '/' sigs{i, 1}], 'PortHandles');
    set_param(ph.Outputport(1), 'DataLogging', 'on', ...
        'DataLoggingNameMode', 'Custom', 'DataLoggingName', sigs{i, 2});
end
set_param(mdl, 'SignalLogging', 'on', 'SignalLoggingName', 'logout');

outA = sim(mdl);
logsA = outA.get('logout');
plotResponse(logsA, 'Nominal operation', fullfile(figDir, 'fig2_nominal_response.png'));
reportNumbers(logsA, 'Nominal');

set_param([mdl '/Bwd_Switch'], 'PhaseDelay', '0');
outB = sim(mdl);
logsB = outB.get('logout');
plotResponse(logsB, 'Fault scenario (both switches pressed)', fullfile(figDir, 'fig3_fault_response.png'));
reportNumbers(logsB, 'Fault');

disp('Figures written to the figures folder.');
```

```

function plotResponse(logs, titleText, fileName)
    v = @(n) logs.getElement(n).Values;
    fwd = v('fwd_log'); bwd = v('bwd_log'); pos = v('pos_log');
    cmd = v('cmd_log'); en = v('en_log'); flt = v('fault_log');

    f = figure('Position', [100 100 900 900], 'Color', 'w', 'Visible', 'off');
    if ~isMATLABReleaseOlderThan('R2025a')
        theme(f, 'light');
    end

    subplot(5, 1, 1);
    stairs(fwd.Time, double(fwd.Data), 'LineWidth', 1.5); hold on;
    stairs(bwd.Time, double(bwd.Data), '--', 'LineWidth', 1.5);
    grid on; ylim([-0.2 1.6]); ylabel('Switch');
    legend('Forward', 'Backward', 'Location', 'north', 'Orientation', 'horizontal');
    title(['Driver inputs - ' titleText]);

    subplot(5, 1, 2);
    plot(pos.Time, pos.Data, 'LineWidth', 1.5, 'Color', [0 0.45 0.74]);
    grid on; ylim([-5 105]); ylabel('mm'); title('Seat position');

    subplot(5, 1, 3);
    plot(cmd.Time, cmd.Data, 'LineWidth', 1.5, 'Color', [0.85 0.33 0.1]);
    grid on; ylim([-1.3 1.3]); ylabel('Command'); title('Motor command after comfort ramp');

    subplot(5, 1, 4);
    stairs(en.Time, double(en.Data), 'LineWidth', 1.5, 'Color', [0.2 0.6 0.2]);
    grid on; ylim([-0.2 1.2]); ylabel('Enable'); title('Motor enable');

    subplot(5, 1, 5);
    stairs(flt.Time, double(flt.Data), 'LineWidth', 1.5, 'Color', [0.8 0 0]);
    grid on; ylim([-0.2 1.2]); ylabel('Fault'); xlabel('Time (s)'); title('Fault flag');

    exportgraphics(f, fileName, 'Resolution', 200);
    close(f);
end

```

```

function reportNumbers(logs, label)
    pos = logs.getElement('pos_log').Values;
    cmd = logs.getElement('cmd_log').Values;
    flt = logs.getElement('fault_log').Values;
    tFront = pos.Time(find(pos.Data >= 100 - 1e-6, 1));
    if isempty(tFront), tFront = NaN; end
    dt = mean(diff(cmd.Time));
    fprintf('%s] front limit reached at t = %.2f s\n', label, tFront);
    fprintf('%s] final position = %.2f mm, min = %.2f, max = %.2f\n', ...
        label, pos.Data(end), min(pos.Data), max(pos.Data));
    fprintf('%s] max command change per step = %.4f\n', label, max(abs(diff(cmd.Data))));
    fprintf('%s] fault active for %.2f s\n', label, sum(logical(flt.Data)) * dt);
end

```